

Exercício de Fixação — Tipos de Dados

Slide 21 & 23

Aluno: Rodrigo Del Padre

RA: 24022092-2

Curso: Engenharia de Software (6º Semestre)

Turma: B

Disciplina: Conceitos / Paradigmas de Linguagens

Referência: R. Sebesta, 11ª ed., Cap. 6

Questão 1 — JavaScript (Ponto Flutuante e Limite de Inteiros)

Código analisado (q1.js):

```
console.log(0.1 * 3 === 0.3);  
console.log(9007199254740993);
```

(a) Saída do programa:

```
false  
9007199254740992
```

(b) Quando o problema é detectado?

NUNCA DETECTADO — O ambiente executa até o final sem qualquer aviso ou exceção, produzindo valores incorretos silenciosamente.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.2.1.2):

Conceito Central: Frações Periódicas Binárias IEEE 754 de 64 bits e Resolução Máxima de Inteiros Seguros (*Safe Integers*).

- **Ponto Flutuante Binário:** O valor decimal 0.1 não possui representação binária exata (dízima periódica em base 2: 0.0001100110011...). Multiplicado por 3 resulta em 0.3000000000000004, falhando na comparação estrita com 0.3.
- **Estouro de Mantissa (53 bits):** O limite de inteiros consecutivos exatos é `Number.MAX_SAFE_INTEGER` ($2^{53} - 1 = 9.007.199.254.740.991$). O literal 9007199254740993 não tem bit correspondente na mantissa e sofre arredondamento IEEE 754 silencioso para o par mais próximo (9007199254740992).
- **Engenharia de Software:** Aplicações financeiras nunca devem usar ponto flutuante binário para valores monetários; adota-se `BigDecimal` ou contagem inteira de centavos.

Questão 2 — Python (Cadeias de Caracteres: Unicode vs. Bytes)

Código analisado (q2.py):

```
palavra = "maçã"
print(len(palavra), len(palavra.encode())) # encode() usa UTF-8 por padrão
```

(a) Saída do programa:

4 6

(b) Quando o problema é detectado?

SEM PROBLEMA / NENHUM ERRO — Comportamento formal e nominal do modelo de tipos do Python 3.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.2.3 e 6.3):

Conceito Central: Abstração Textual (Pontos de Código Unicode) vs. Representação Binária Física (Octetos em UTF-8).

- **Comprimento de Caracteres:** `len(palavra)` afere a quantidade de caracteres abstratos (pontos de código Unicode): "m", "a", "ç", "ã" = **4 caracteres**.
- **Comprimento em Bytes:** A codificação padrão UTF-8 usa 1 byte para caracteres ASCII ('m', 'a') e 2 bytes para caracteres acentuados ('ç' = `\xc3\xa7`, 'ã' = `\xc3\xa3`). Total: $1 + 1 + 2 + 2 = \mathbf{6 \text{ bytes}}$.
- **Impacto Prático:** Dimensionar colunas de banco ou buffers de rede com base apenas na quantidade de caracteres acarreta truncamento de dados com textos em UTF-8.

Questão 3 — Go (Inteiros Sem Sinal e Aritmética Modular)

Código analisado (q3.go):

```
package main
import "fmt"

func main() {
    var b byte = 255
    b++
    fmt.Println(b)
}
```

(a) Saída do programa:

0

(b) Quando o problema é detectado?

NUNCA DETECTADO — Estouro de inteiro (*overflow*) silencioso por aritmética modular sem pânico da runtime.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.2.1.1):

Conceito Central: Aritmética Modular de Inteiros de Tamanho Fixo e Eficiência de Instrução de Máquina.

- **Tipo byte (uint8):** Em Go, `byte` é sinônimo de `uint8`, ocupando 8 bits sem sinal com faixa $[0, 255]$.
- **Estouro Silencioso:** Ao incrementar além de 255, a operação em runtime obedece à aritmética modular ($255 + 1 \equiv 0 \pmod{256}$). Go não insere checagens de carry/overflow em runtime para operações com variáveis, priorizando desempenho bruto.
- **Segurança:** Estouro de inteiros silencioso é vetor clássico de vulnerabilidades (CWE-190), podendo anular validações de limites em buffers.

Questão 4 — Java (Matrizes, Inicialização Padrão e Limites)

Código analisado (Q4.java):

```
public class Main {  
    public static void main(String[] args) {  
        int[] v = new int[3];  
        System.out.println(v[0]);  
        System.out.println(v[3]);  
    }  
}
```

(a) Saída do programa:

```
0  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for  
length 3
```

(b) Quando o problema é detectado?

NA EXECUÇÃO (Runtime) — Compila sem erros, imprime o primeiro elemento inicializado e aborta ao acessar o índice inválido.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.5):

Conceito Central: Alocação Dinâmica no Heap com Inicialização Neutra e Checagem Dinâmica de Limites (*Subscript Bounds Checking*).

- **Inicialização Automática:** Vetores em Java são objetos alocados no heap. Seus elementos primitivos são inicializados com zero por padrão, imprimindo 0 para `v[0]`.
- **Verificação de Limites:** Em um array de tamanho 3, os índices legais são exclusivamente 0, 1, 2. O acesso a `v[3]` ultrapassa os limites.
- **Prevenção de Corrupção:** Ao contrário de C (que leria memória adjacente com ponteiro descontrolado), a JVM verifica os limites a cada acesso e lança `ArrayIndexOutOfBoundsException`, impedindo invasão de memória.

Questão 5 — Rust (Semântica de Posse, Movimento e Borrow Checker)

Código analisado (q5.rs):

```
fn main() {  
    let s = String::from("oi");  
    let t = s;  
    println!("{}", s, t);  
}
```

(a) Saída do programa:

```
error[E0382]: borrow of moved value: `s`  
--> q5.rs:5:23  
| let t = s; // value moved here  
| println!("{}", s, t); // value borrowed here after move
```

(b) Quando o problema é detectado?

NA COMPILAÇÃO (Compile-time) — O compilador rustc barra o código antes da geração do binário.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.11):

Conceito Central: Semântica de Movimento (*Move Semantics*), Propriedade Única (*Ownership*) e Empréstimos (*Borrowing*).

- **Posse Única de Heap:** O tipo `String` gerencia memória no heap e não implementa o trait `Copy`.
- **Movimento de Posse:** A instrução `let t = s;` transfere a propriedade exclusiva do buffer para `t` e invalida `s` na pilha para prevenir liberação dupla (*double free*).
- **Borrow Checker:** Ao tentar ler `s` em `println!`, o verificador de empréstimos acusa que o valor já foi movido. O erro é erradicado no desenvolvimento sem nenhum overhead em tempo de execução.

Questão 6 — C (Unições Livres e Corrupção da Tipagem Estática)

Código analisado (q6.c):

```
#include <stdio.h>

int main(void) {
    union { int i; float f; } u;
    u.f = 1.0f;
    printf("%d\n", u.i);
    return 0;
}
```

(a) Saída do programa:

1065353216

(b) Quando o problema é detectado?

NUNCA DETECTADO — O processador lê os bits crus do float como inteiro; nenhuma inconsistência de tipo é acusada.

(c) Análise Conceitual e Fundamentação Teórica (Sebesta 6.10):

Conceito Central: Unições Livres (*Free Unions*) e Quebra de Segurança de Tipos (*Type Safety*).

- **Compartilhamento Físico de Memória:** Os membros `i` e `f` residem no mesmo endereço de 4 bytes.
- **Reinterpretação Binária IEEE 754:** O float `1.0f` possui padrão binário `0x3F800000` (sinal 0, expoente 127, mantissa 0). Ao ler como `int` em complemento de dois, `0x3F800000 = 1065353216`.
- **Violação de Tipagem:** Como C usa uniões sem discriminadores de tipo ativo, a segurança de tipos é anulada, criando vulnerabilidades de exploração de memória.

Síntese Comparativa: Análise de Trade-off (Slides 21 e 22)

Momento da Detecção	Exemplos / Linguagens	Vantagens	Custos / Desvantagens
1. Na Compilação <i>(Mais precoce)</i>	Rust (q5.rs — Erro E0382)	• Eliminação total de bugs antes de produção. • Zero custo em runtime. • Segurança de memória sem GC.	• Compilação mais lenta. • Curva de aprendizado íngreme. • Maior esforço para codificar.
2. Na Execução <i>(Intermediário)</i>	Java (Q4.java — Exceção de Limites)	• Bloqueia leitura de lixo de memória adjacente. • Exceções rastreáveis e fáceis de depurar.	• Ciclos de CPU extras para checar limites a cada acesso. • Risco de crash se a exceção não for tratada.
3. Nunca Detectado <i>(Mais tardio)</i>	C (união livre) Go (estouro modular) JavaScript (IEEE 754)	• Desempenho máximo de máquina (C). • Simplicidade arquitetural do compilador.	• Bugs lógicos silenciosos em cálculos críticos. • Vulnerabilidades de segurança da informação (CWE).

Autopesquisa (Slide 23): Tipos de Dados Avançados

Linguagem	Matriz Associativa (Mapa / Dict)	Registros (Structures / Records)	Tuplas
Java	Map<K,V> (HashMap, TreeMap). Chaves e valores são referências a objetos no heap.	record (Java 16+). Registro imutável conciso com getters, equals, hashCode e toString gerados.	Sem tupla nativa primitiva. Implementada com records utilitários de 2 campos.
Python	dict ({ "k": v}). Otimizado em C. Mantém a ordem de inserção garantida desde a versão 3.7.	class, @dataclass ou NamedTuple. Campos heterogêneos mutáveis por padrão.	tuple ((1, "x", True)). Coleção ordenada e estritamente imutável com desempacotamento nativo.
Go	map[K]V. Ordem de iteração é deliberadamente randomizada pela runtime para evitar acoplamento.	struct. Composição contígua de campos nomeados com controle de alinhamento de memória.	Sem tipo de tupla direto. Retornos múltiplos de função simulam tuplas.
Rust	HashMap<K,V>. Usa por padrão o algoritmo SipHash para resistência a ataques de DoS.	struct. Controle rigoroso de layout, alinhamento e tempo de vida; imutável por padrão.	(T1, T2, ...). Tipo primitivo heterogêneo de tamanho fixo alocado na pilha com pattern matching.